V2 / LEARNING GUIDE

Adding a backend, email, and distributed tracing

aldhairmartinez.com

A practical record of V2: FastAPI, Docker Compose, Resend, Cloudflare Email Routing, OpenTelemetry, Grafana Alloy, Tempo, Application Observability, and browser-to-backend trace propagation.

V2 DEFINITION A real Python/FastAPI backend running locally in Docker, connected to the Next.js frontend, capable of sending contact email, instrumented with OpenTelemetry, routed through Grafana Alloy, and verified end-to-end in Grafana Cloud.

```
LOCAL V2
Browser / Next.js :3000
        |  POST /api/contact + traceparent
        v
FastAPI :8000  --->  Resend API  --->  email
        |  OTLP/HTTP
        v
Grafana Alloy :4318
        |
        v
Grafana Cloud  --->  Tempo + Application Observability
```

Build -> Connect -> Instrument -> Observe -> Debug -> Document

V2 keeps the production frontend on Cloudflare Pages. The backend and Alloy are real and working, but remain local-only until a later AWS phase.

V2 at a glance

Concept	In this project
Frontend	Next.js static frontend; production remains on Cloudflare Pages
Backend	Python + FastAPI, running locally in Docker Desktop
API	GET /health, GET /version, POST /api/contact
Email	Resend API for outbound contact notifications; Cloudflare Email Routing for hello@aldhairmartinez.com inbound forwarding
Containers	backend + Grafana Alloy managed with Docker Compose
Backend telemetry	OpenTelemetry FastAPI + HTTPX instrumentation
Collector	Grafana Alloy receives OTLP/HTTP on port 4318
Trace backend	Grafana Cloud Tempo
App view	Grafana Cloud Application Observability
Distributed tracing	Faro browser trace context -> FastAPI -> Resend
Production status	Frontend/Faro live; FastAPI/Alloy live locally, not production-hosted

KEY MENTAL MODEL V2 adds request-time server code. The browser can now call an API, the API can call external services, and one trace can follow that request across multiple layers.

The frontend/backend boundary

V1 proved the site could be production-ready without a backend. V2 deliberately adds one so the project can perform request-time work instead of only serving prebuilt files.

```
LOCAL DEVELOPMENT

Browser
  |
  +--> http://localhost:3000  -> Next.js dev server
  |
  +--> http://localhost:8000  -> FastAPI container

PRODUCTION TODAY

Browser -> https://aldhairmartinez.com -> Cloudflare Pages -> static files

There is still no production FastAPI server yet.
```

Concept	In this project
Port 3000	Local Next.js development server. It does not exist as a Next.js server in production because Cloudflare serves the static export.
Port 8000	Local doorway into the FastAPI/Uvicorn server running in Docker.
API	A contract that lets the browser request server-side work using HTTP.
JSON	The structured request/response format used by the contact endpoint.
CORS	Browser security rules deciding which frontend origins may call FastAPI.
LAN IP	The Mac's private Wi-Fi address, used so an iPhone on the same network can reach the Mac instead of its own localhost.

WHY THE IPHONE NEEDED A LAN IP localhost always means “this device.” On the iPhone, localhost would point back to the iPhone. The Mac LAN address let the phone reach the development servers over the home network.

FastAPI, Uvicorn, and the first API

```
GET  /health       -> {"status":"ok"}
GET  /version      -> {"version":"0.1.0"}
POST /api/contact  -> validate JSON -> send email -> return success/error
```

Concept	In this project
FastAPI	Python web framework defining routes, request models, validation, and responses.
Uvicorn	The ASGI web server process that actually listens for HTTP traffic on port 8000.
/health	Simple liveness-style endpoint: “is the service responding?”
/version	Returns the application version so the running build can be identified.
/api/contact	Real application work: validate visitor input and call Resend.
422 response	FastAPI validation failure for malformed/invalid input.
5xx response	Server-side failure path, such as an external email-provider problem.

WHY THESE ENDPOINTS They teach three useful backend patterns: service health, deployment identity, and a real POST workflow with validation and an external dependency.

Docker and Docker Compose

V2 runs the backend in a container instead of depending on the Mac's Python environment. Docker Compose then gives the project a repeatable way to run multiple cooperating services.

```
docker-compose.yml

backend
  FastAPI / Uvicorn
  OTLP endpoint = http://alloy:4318

alloy
  OTLP receiver :4318
  batch processor
  Grafana Cloud exporter
```

Concept	In this project
Dockerfile	Recipe used to build the backend image: base Python image, dependencies, application files, startup command.
Image	Packaged blueprint produced from the Dockerfile.
Container	A running instance of an image.
Compose	Defines and starts multiple services together.
Docker network	Compose creates a private network and DNS. The backend can resolve the hostname alloy automatically.
alloy:4318	Container-to-container address for OTLP/HTTP traces. No public internet hostname is needed between these local services.

WHY ALLOY BECAME USEFUL Direct SDK-to-cloud export was simpler on paper, but Alloy gives the lab a central telemetry layer and keeps Grafana Cloud credentials out of the application container.

Contact email: receiving vs sending

V2 introduced two separate email concerns that are easy to confuse: receiving mail for the custom domain and sending application-generated mail.

```
INBOUND MAIL
Someone -> hello@aldhairmartinez.com -> Cloudflare Email Routing -> personal Gmail

CONTACT FORM
Browser -> FastAPI -> Resend -> configured destination inbox
             From: noreply@aldhairmartinez.com
             Reply-To: visitor email
```

Concept	In this project
Cloudflare Email Routing	Receives mail addressed to hello@aldhairmartinez.com and forwards it to the existing Gmail inbox.
Resend	External email API used by FastAPI to send contact-form notifications.
From	noreply@aldhairmartinez.com, authenticated through the verified domain.
Reply-To	The visitor's submitted email, so Gmail Reply targets the person rather than noreply.
API key	Secret stored only in backend/.env; never committed.
Future mailbox	Google Workspace is planned later if hello@aldhairmartinez.com becomes a full standalone mailbox.

IMPORTANT DISTINCTION Forwarding gives the custom address an inbound route without creating a separate mailbox. A full mailbox would have its own login, storage, sent mail, and mailbox provider.

OpenTelemetry and Grafana Alloy

The backend uses upstream OpenTelemetry instrumentation. FastAPI request spans and HTTPX outbound-call spans are created in-process, then exported with OTLP to Alloy.

```
FastAPI
  | OpenTelemetry spans
  | OTLP/HTTP
  v
Grafana Alloy
  | receive -> batch -> authenticate -> export
  v
Grafana Cloud OTLP gateway
  |
  +--> Tempo
  +--> Application Observability
```

Concept	In this project
OpenTelemetry	Vendor-neutral APIs, SDKs, semantic conventions, and protocols for telemetry.
OTLP	The transport protocol used to move OTel telemetry. OTLP is not the trace database.
Tempo	The Grafana trace backend where traces are stored and queried.
Alloy	Collector/processing layer between applications and Grafana Cloud.
FastAPI instrumentation	Automatically creates spans for incoming server requests.
HTTPX instrumentation	Automatically creates child spans for outbound HTTP calls such as Resend.
Batching	Alloy groups telemetry before export rather than sending every span independently.

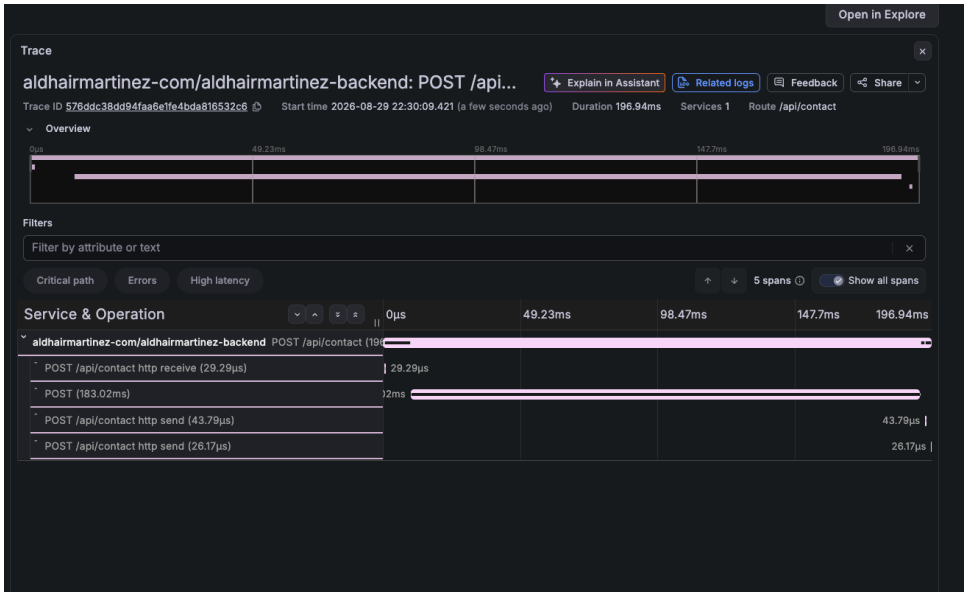
MENTAL MODEL OTLP is how traces travel. Alloy is the collector. Tempo is where traces land. Application Observability is a higher-level service view built from the telemetry.

One distributed trace: browser -> backend -> Resend

V2 finished the tracing story by connecting the browser trace context to the FastAPI request. Faro adds W3C trace context to the cross-origin request; FastAPI continues that context instead of starting a new unrelated trace.

```
Browser / Faro
  | traceparent
  v
POST /api/contact   [FastAPI server span]
  |
  +--> POST api.resend.com/emails   [HTTPX client span]
  |
  v
Alloy -> Grafana Cloud -> Tempo
```

Concept	In this project
traceparent	W3C header carrying trace identity and parent-span context across the HTTP boundary.
propagateTraceHeaderCorsUrls	Faro configuration controlling which cross-origin requests may receive trace headers.
CORS headers	FastAPI must allow traceparent/tracestate so the browser preflight permits them.
Same trace ID	Evidence that FastAPI continued the browser trace rather than starting a new root trace.
Resend child span	HTTPX instrumentation nests the external email call under the FastAPI request.



Verified Tempo trace for POST /api/contact. The outbound Resend call accounts for most of the request latency.

WHAT THE TRACE PROVED The request completed successfully in about 197 ms, and the external Resend call consumed about 183 ms. The trace made the dependency cost visible without guessing from application code.

Security, secrets, and environment configuration

Concept	In this project
backend/.env	Local FastAPI secrets/config, including the Resend API key. Gitignored.
alloy/.env	Local Grafana Cloud endpoint/credential values for Alloy. Gitignored.
.env.local	Local Next.js browser-visible configuration. Gitignored.
.env.example files	Committed templates that document variable names but never real secrets.
NEXT_PUBLIC_*	Browser-visible by design. Never use for secrets.
Access Policy token	Secret credential. Use least privilege; the collector only needs the permissions required to write the intended telemetry.

SECURITY LESSON A secret that appears in logs or tool output should be treated as compromised and rotated. During V2, a Grafana token was accidentally printed by low-level HTTP debug logging; the correct response was revoke, replace, and stop logging sensitive headers.

Alloy also simplified credential ownership: FastAPI only knows the local collector address. Alloy owns the cloud authentication boundary.

Debugging lessons from V2

Concept	In this project
iPhone form looked right but did not submit	Next.js dev-origin protection served initial HTML but blocked the JavaScript chunks. The browser fell back to a normal HTML form GET. Fix: allow the LAN development origin.
Faro rejected LAN telemetry	Grafana allowed localhost and the production domain, but the LAN-hosted development copy was a different origin. Fix: intentionally skip Faro on arbitrary LAN origins rather than polluting production allowed origins.
Faro internalLogger error	Development remount/double-initialization path. Fix: guard initialization using Faro's persistent window marker before initializing again.
Direct OTLP 401	Authentication/header formatting became fragile when the app tried to own Grafana Cloud auth directly. Architecture changed to FastAPI -> Alloy.
Alloy auth confusion	Python-generated combined OTLP header was not the cleanest input for Alloy. Final design uses separate endpoint, Instance ID, and token; Alloy builds Basic Auth.
Trace latency	Tempo showed Resend, not FastAPI code, dominated /api/contact latency.

ENGINEERING LESSON The strongest debugging steps used evidence from the actual layer that failed: browser network logs, server logs, collector metrics, HTTP status codes, and traces. “The code looks right” was never treated as proof.

Command + concept cheat sheet

Concept	In this project
npm run dev	Start the local Next.js development server on port 3000.
docker compose up --build	Build/start the local backend and Alloy services.
docker compose down	Stop/remove the Compose services.
http://localhost:8000/docs	FastAPI-generated interactive API documentation.
curl /health	Simple direct API reachability test.
OTLP/HTTP 4318	Standard HTTP/protobuf OTLP receiver port used between FastAPI and Alloy.
traceparent	W3C header used to continue a distributed trace across HTTP.
service.name	Primary service identity used by traces and Application Observability.
service.namespace	Groups related services; V2 uses aldhairmartinez-com.
deployment.environment	Separates development from later production telemetry.

What V2 taught

- A frontend and backend are separate runtime systems even when they live in the same repository.
- Ports are local network entry points; production static hosting does not imply a Next.js server on port 3000.
- Docker images are blueprints; containers are running instances; Compose coordinates multiple services and their network.
- CORS is about website origins, not about approving individual users or their IP addresses.
- A custom-domain forwarding address and a full mailbox are different email architectures.
- External APIs should be treated as dependencies with explicit failure handling and observability.
- OpenTelemetry creates and propagates telemetry; OTLP transports it; Alloy collects/processes it; Tempo stores traces.
- Distributed tracing becomes useful when trace context crosses real service boundaries.
- Application Observability and raw distributed traces can use the same underlying backend telemetry.
- Secrets belong in environment-specific secret stores/files, not source control or browser-visible variables.
- Collector architecture centralizes credentials and gives future services a common telemetry path.
- Observability is most useful when it answers a concrete question, such as “where did the contact request spend its time?”

V2 COMPLETE The local backend is real, email works, traces flow through Alloy, Application Observability recognizes the service, and browser-to-backend trace context is verified. The backend is intentionally not production-hosted yet.

What comes next: V3 data layer

V3 adds PostgreSQL one use case at a time so the database exists for real application reasons rather than as a technology checkbox.

Concept	In this project
1. Contact history	Persist contact-form submissions in PostgreSQL in addition to sending the email.
2. Resume analytics	Record PDF/DOCX download events and useful request metadata.
3. Project analytics	Record project views/interactions and query aggregate behavior.
4. Deployment history	Store application/release metadata and surface it on /observability.
Redis later	Add only when a real caching/use-case reason exists.
Kafka later	Add only when asynchronous/event processing has a real job.
AWS later	Move FastAPI/Alloy from Docker Desktop to real production infrastructure.
Google Workspace later	Turn hello@aldhairmartinez.com from forwarding into a full custom-domain mailbox.

RULE FOR THE LAB Build one thing -> understand it -> instrument it -> verify it -> document it -> then add the next layer.

V2 complete: backend + email + collector + distributed tracing verified locally end-to-end.